

Distributional Reinforcement Learning | Part 2

Jonas Müller

June 26, 2026

Outline

- 1 Recap
- 2 Scalar Linear Function Approximation
- 3 Deep Distributional Reinforcement Learning

Recap: MDP and Return

We are working on a finite MDP:

$$(\mathcal{X}, \mathcal{A}, \xi_0, P_X, P_R), \quad \pi : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{A}), \quad \gamma \in [0, 1).$$

Trajectory process:

$$\begin{aligned} X_0 &\sim \xi_0, & A_t &\sim \pi(\cdot \mid X_t), \\ R_t &\sim P_R(\cdot \mid X_t, A_t), & X_{t+1} &\sim P_X(\cdot \mid X_t, A_t), \end{aligned} \quad t \geq 0.$$

Discounted return:

$$G^\pi(x) = \sum_{t=0}^{\infty} \gamma^t R_t \quad \text{given } X_0 = x.$$

Recap: Value function and Bellman Operator

The value function is given by the expected return:

$$V^\pi(x) = \mathbb{E}[G^\pi(x)] = \int_{\mathbb{R}} z \, d\eta_\pi(x)(z)$$

Where $\eta_\pi(x) = \mathcal{D}(G^\pi(x))$ is the return distribution.

As we know, there is a recursive relation for the value function known as Bellman's Equation:

$$V^\pi = \mathcal{T}^\pi V^\pi.$$

Where we have the Bellman operator $\mathcal{T}^\pi : \mathbb{R}^{\mathcal{X}} \rightarrow \mathbb{R}^{\mathcal{X}}$:

$$(\mathcal{T}^\pi V)(x) = \mathbb{E}_\pi [R + \gamma V(X') \mid X = x].$$

Recap: Random-Variable Bellman Equation

With (A, R, X') sampled from $\pi(\cdot | x)P_R(\cdot | x, A)P_X(\cdot | x, A)$, the random-variable Bellman equation is:

$$G^\pi(x) \stackrel{D}{=} R + \gamma G^\pi(X') \mid X = x.$$

Taking laws gives the return-distribution equation:

$$\eta_\pi(x) = \mathcal{D}(R + \gamma G^\pi(X') \mid X = x).$$

This is the distributional version of $(\mathcal{T}^\pi V)(x) = \mathbb{E}_\pi[R + \gamma V(X') \mid X = x]$.

Recap: Distributional Bellman Operator

Define the bootstrap map

$$b_{r,\gamma}(z) = r + \gamma z, \quad r, z \in \mathbb{R},$$

and for $Z' \sim \eta(x')$,

$$(b_{r,\gamma})_{\#}\eta(x') = \mathcal{D}(r + \gamma Z').$$

Distributional Bellman operator $\mathcal{T}^{\pi} : \mathcal{P}(\mathbb{R})^{\mathcal{X}} \rightarrow \mathcal{P}(\mathbb{R})^{\mathcal{X}}$:

$$(\mathcal{T}^{\pi}\eta)(x) = \mathbb{E}_{\pi} [(b_{R,\gamma})_{\#}\eta(X') \mid X = x].$$

Distributional Bellman equation:

$$\eta_{\pi}(x) = \mathbb{E}_{\pi} [(b_{R,\gamma})_{\#}\eta_{\pi}(X') \mid X = x], \quad \eta_{\pi} = \mathcal{T}^{\pi}\eta_{\pi}.$$

Recap: Representations of Return Distributions

Categorical representation:

$$\eta(x) = \sum_{i=1}^m p_i^\theta(x) \delta_{\theta_i}, \quad p_i^\theta(x) \geq 0, \quad \sum_{i=1}^m p_i^\theta(x) = 1.$$

Quantile representation:

$$\eta(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x)}, \quad \tau_i = \frac{2i-1}{2m}, \quad \theta_i(x) \approx F_{\eta(x)}^{-1}(\tau_i).$$

Focus of this Presentation

If $\mathcal{X}$ is large, then storing even a scalar, let alone a distribution per state is expensive. Hence, we look for suitable approximations.

Linear Value Function Approximation | Scalar Case

For weights $w \in \mathbb{R}^n$ approximate the value function as

$$V_w(x) = \phi(x)^\top w, \quad \nabla_w V_w(x) = \phi(x).$$

For finite $\mathcal{X}$ and feature map ϕ , stack feature rows into $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$:

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For state-action pairs, use e.g. $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$ via $Q_w(x, a) = \phi(x, a)^\top w$.

Idea

Instead of having a lookup table for the value of each state, we use a linear function to represent the function. **This function can be queried cheaply.**

Linear Value Function Approximation | Scalar Case

For weights $w \in \mathbb{R}^n$ approximate the value function as

$$V_w(x) = \phi(x)^\top w, \quad \nabla_w V_w(x) = \phi(x).$$

For finite $\mathcal{X}$ and feature map ϕ , stack feature rows into $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$:

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For state-action pairs, use e.g. $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$ via $Q_w(x, a) = \phi(x, a)^\top w$.

Idea

Instead of having a lookup table for the value of each state, we use a linear function to represent the function. **This function can be queried cheaply.**

Question: How do we get a good w ?

Linear Approximation as Regression

Fitting a linear value function is just ordinary least squares:

$$V_w = \Phi w, \quad \min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_2.$$

which we could solve by the normal equation.

However

In reinforcement learning, we might want to have a better approximation for some states than others.

Weighted Norm for Linear Approximation

For some $\xi \in \mathcal{P}(\mathcal{X})$ we define the weighted L_2 norm and diagonal weight matrix

$$\|V - V'\|_{\xi,2} = \left(\sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}. \quad \Xi = \text{diag}(\xi(x) : x \in \mathcal{X}).$$

The problem we now want to solve is:

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

Weighted Norm for Linear Approximation

For some $\xi \in \mathcal{P}(\mathcal{X})$ we define the weighted L_2 norm and diagonal weight matrix

$$\|V - V'\|_{\xi,2} = \left(\sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}. \quad \Xi = \text{diag}(\xi(x) : x \in \mathcal{X}).$$

The problem we now want to solve is:

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

Proposition 9.3 | Weighted Normal Equation

If ξ has full support and the columns of Φ are linearly independent, the unique solution is

$$w^* = (\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi V^\pi.$$

Projection Back into the Feature Space

We have a linear approximation now, however we can't iterate the Bellman operator because we could have

$$\mathcal{T}^\pi V_w \notin \mathcal{F}_\phi$$

Define the weighted orthogonal projection

$$(\Pi_{\phi,\xi} V)(x) = \phi(x)^\top w^*, \quad w^* \in \arg \min_w \|V - V_w\|_{\xi,2}.$$

Approximate dynamic programming therefore iterates

$$V_{k+1} = \Pi_{\phi,\xi} \mathcal{T}^\pi V_k.$$

- $\mathcal{T}^\pi$ performs the Bellman update.
- $\Pi_{\phi,\xi}$ removes components that the features cannot express.

Repeated projection can compound error (the diffusion effect).

When Is the Projected Operator Stable?

Let ξ_π be a fully supported steady-state distribution under π .

Projected Bellman contraction / Theorem 9.8

Since P^π and Π_{ϕ, ξ_π} are nonexpansions in $\|\cdot\|_{\xi_\pi, 2}$,

$$\|\Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V - \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V'\|_{\xi_\pi, 2} \leq \gamma \|V - V'\|_{\xi_\pi, 2}.$$

Therefore there is a unique fixed point $\hat{V}^\pi = \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi \hat{V}^\pi$, with

$$\|\hat{V}^\pi - V^\pi\|_{\xi_\pi, 2} \leq \frac{1}{\sqrt{1 - \gamma^2}} \|\Pi_{\phi, \xi_\pi} V^\pi - V^\pi\|_{\xi_\pi, 2}.$$

Scalar Linear Approximation: Proof Chain

The scalar argument works because everything lives in a Hilbert space:

$$\mathbb{R}^{\mathcal{X}}, \quad \langle V, V' \rangle_{\xi_\pi} = \sum_{x \in \mathcal{X}} \xi_\pi(x) V(x) V'(x).$$

The feature class $\mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}$ is a linear subspace.

- ❶ Π_{ϕ, ξ_π} is an orthogonal projection, hence nonexpansive.
- ❷ Stationarity of ξ_π makes P^π nonexpansive in $\|\cdot\|_{\xi_\pi, 2}$.
- ❸ The Bellman operator is affine with linear part γP^π .

Therefore

$$\Pi_{\phi, \xi_\pi} \mathcal{T}^\pi \quad \text{is a } \gamma\text{-contraction.}$$

Monte Carlo as Stochastic Gradient Descent

Given independent state–return samples (x_k, g_k) , use the sample loss

$$L(w) = (g_k - \phi(x_k)^\top w)^2.$$

Absorbing the factor 2 into the step size gives

$$w \leftarrow w + \alpha_k (g_k - \phi(x_k)^\top w) \phi(x_k).$$

This is genuine stochastic gradient descent: the target g_k does not depend on w . Under standard sampling and step-size conditions, it approaches the weighted least-squares solution.

Semi-Gradient Temporal-Difference Learning

For a transition (x_k, a_k, r_k, x'_k) , replace the Monte Carlo return by the bootstrapped target

$$r_k + \gamma V_w(x'_k) = r_k + \gamma \phi(x'_k)^\top w.$$

The update is

$$w \leftarrow w + \alpha_k \underbrace{\left(r_k + \gamma \phi(x'_k)^\top w - \phi(x_k)^\top w \right)}_{\text{TD error}} \phi(x_k).$$

It is a *semi*-gradient: while differentiating, the target is treated as fixed. Differentiating the full squared Bellman error would also produce a term involving $\phi(x'_k)$.

Target Weights Explain the Projection Connection

Freeze a second vector $\tilde{w}$ inside the target:

$$w \leftarrow w + \alpha_k (r_k + \gamma \phi(x'_k)^\top \tilde{w} - \phi(x_k)^\top w) \phi(x_k).$$

This is SGD on a fixed-target loss and approximately computes one projected Bellman step:

$$\Phi w^* = \Pi_{\phi, \xi} \mathcal{T}^\pi V_{\tilde{w}}.$$

Setting $\tilde{w} = w$ recovers ordinary semi-gradient TD, but also creates a feedback loop. Function approximation, bootstrapping, and an unsuitable sampling distribution can then produce divergence (Baird's counterexample).

Why “Linear Distribution” Needs Care

Probability distributions do not form a vector space:

$$\eta_{w_1+w_2}(x) \neq \eta_{w_1}(x) + \eta_{w_2}(x)$$

because the right side has total mass 2.

Book's convention

“Linear” distributional approximation means that the *parameters of a probability representation* depend linearly on features.

There are therefore two approximation layers:

- 1 a finite probability representation (categorical or quantile),
- 2 feature aliasing across states.

Linear Quantile TD: Representation and Loss

With m quantile locations and weights $w_1, \dots, w_m \in \mathbb{R}^n$,

$$\eta_w(x) = \frac{1}{m} \sum_{i=1}^m \delta_{\phi(x)^\top w_i}, \quad \tau_i = \frac{2i-1}{2m}.$$

Define the quantile loss

$$\rho_\tau(\Delta) = |\mathbb{1}_{\{\Delta < 0\}} - \tau| |\Delta|.$$

For one target z , head i uses

$$L_{\tau_i}(w_i) = \rho_{\tau_i}(z - \phi(x)^\top w_i),$$

whose subgradient is proportional to $-(\tau_i - \mathbb{1}_{\{z < \phi(x)^\top w_i\}})\phi(x)$.

Linear QTD: Distributional Semi-Gradient

A transition (x, a, r, x') supplies m target samples

$$g_j = r + \gamma \phi(x')^\top w_j, \quad j = 1, \dots, m.$$

Averaging the quantile subgradient over these targets yields

$$w_i \leftarrow w_i + \alpha \frac{1}{m} \sum_{j=1}^m \underbrace{(\tau_i - \mathbb{1}_{\{g_j < \phi(x)^\top w_i\}})}_{\text{quantile TD error}} \phi(x).$$

As in scalar TD, g_j is treated as constant during differentiation. Unlike the scalar TD error, each averaged quantile error is bounded.

Linear Categorical TD: Prediction

Fix support locations $\theta_1 < \dots < \theta_m$. Linear logits are transformed into probabilities by a softmax:

$$p_i(x; w) = \frac{\exp(\phi(x)^\top w_i)}{\sum_{j=1}^m \exp(\phi(x)^\top w_j)}, \quad \eta_w(x) = \sum_{i=1}^m p_i(x; w) \delta_{\theta_i}.$$

The softmax is essential: arbitrary linear coefficients need not be nonnegative or sum to one.

Linear CTD: Target, Loss, and Update

Project the Bellman-updated next distribution back to the fixed support:

$$\bar{\eta}(x) = \Pi_c(b_{r,\gamma})_{\#} \eta_w(x') = \sum_{i=1}^m \bar{p}_i \delta_{\theta_i}.$$

Cross-entropy is the matching loss for softmax probabilities:

$$L(w) = - \sum_{i=1}^m \bar{p}_i \log p_i(x; w).$$

Its semi-gradient update has the same template as linear TD and QTD:

$$w_i \leftarrow w_i + \alpha \underbrace{(\bar{p}_i - p_i(x; w))}_{\text{CTD error}} \phi(x).$$

Thus all three methods have the form $w_i \leftarrow w_i + \alpha \varepsilon_i \phi(x)$.

Distributional Projection: What Is Projected?

The distributional Bellman update usually leaves the chosen finite family:

$$\mathcal{T}^\pi \eta_w \notin \left\{ \frac{1}{m} \sum_i \delta_{\phi(x)^\top w_i} : w_i \in \mathbb{R}^n \right\}.$$

Hence the conceptual fixed point is again projected:

$$\eta = \Pi_{\mathcal{F}} \mathcal{T}^\pi \eta.$$

The projection is no longer the scalar orthogonal projection $\Pi_{\phi, \xi}$.

- QTD uses a *quantile projection*: choose atom locations by quantile regression.
- CTD uses a *categorical projection*: redistribute mass onto a fixed support.
- Linear features add a second projection/aliasing effect across states.

Chapter 9 Objects: Signed Categorical Family

Fix support locations $S = \{\theta_1 < \dots < \theta_m\}$. The signed categorical family on S is

$$\mathcal{F}_{S,m} = \left\{ \sum_{i=1}^m p_i \delta_{\theta_i} : p_i \in \mathbb{R}, \sum_{i=1}^m p_i = 1 \right\}.$$

Unlike ordinary CTD probabilities, the coefficients p_i may be negative. Over states,

$$\mathcal{F}_{S,m}^{\mathcal{X}} = \{\eta : \mathcal{X} \rightarrow \mathcal{F}_{S,m}\}.$$

The book's clean linear convergence theorem is for this signed categorical variant, not for softmax CTD or QTD directly.

Chapter 9 Objects: Linear Signed Approximation

Let $\psi : \mathcal{X} \rightarrow \mathbb{R}^d$ be a feature map. The book uses signed categorical weights of the form

$$p_i(x) = \frac{1}{m} + \psi(x)^\top w_i - \frac{1}{m} \sum_{j=1}^m \psi(x)^\top w_j.$$

Then

$$\sum_{i=1}^m p_i(x) = 1, \quad p_i(x) \in \mathbb{R}.$$

The resulting linearly representable family is

$$\mathcal{F}_{\psi, S, m} = \left\{ \eta_W : \eta_W(x) = \sum_{i=1}^m p_i(x) \delta_{\theta_i} \in \mathcal{F}_{S, m} \ \forall x \right\}.$$

Chapter 9 Objects: Cramér Geometry

For a signed distribution ν , let

$$F_\nu(z) = \nu((-\infty, z])$$

denote its cumulative distribution function. The Cramér distance is

$$\ell_2^2(\nu, \nu') = \int_{\mathbb{R}} (F_\nu(z) - F_{\nu'}(z))^2 dz$$

whenever this quantity is finite. Define

$$\mathcal{M}_{\ell_2,1}(\mathbb{R})$$

as the set of signed distributions ν with total mass $\nu(\mathbb{R}) = 1$ and finite Cramér distance to the reference distribution δ_0 . For return-distribution functions $\eta : \mathcal{X} \rightarrow \mathcal{M}_{\ell_2,1}(\mathbb{R})$,

$$\ell_{\xi_\pi,2}^2(\eta, \eta') = \sum_{x \in \mathcal{X}} \xi_\pi(x) \ell_2^2(\eta(x), \eta'(x)).$$

Chapter 9 Objects: The Two Projections

The categorical projection maps a signed distribution to the fixed support:

$$\Pi_c : \mathcal{M}_{\ell_2,1}(\mathbb{R}) \rightarrow \mathcal{F}_{S,m}.$$

The feature projection maps signed categorical return functions back into the linearly representable family:

$$\Pi_{\psi,\xi_\pi,\ell_2} : \mathcal{F}_{S,m}^{\mathcal{X}} \rightarrow \mathcal{F}_{\psi,S,m}.$$

It is defined as the $\ell_{\xi_\pi,2}$ -closest element of the linear feature class. The doubly projected Bellman operator is

$$\Pi_{\psi,\xi_\pi,\ell_2} \Pi_c \mathcal{T}^\pi.$$

The order is: Bellman update, categorical projection, feature projection.

Chapter 9 Lemmas: Bellman and Categorical Steps

Assume rewards have finite first moments and the Markov chain under π has a unique stationary distribution ξ_π .

Lemma 9.14: Bellman Step

For any $\eta, \eta' \in \mathcal{M}_{\ell_2,1}(\mathbb{R})^{\mathcal{X}}$,

$$\ell_{\xi_\pi,2}(\mathcal{T}^\pi \eta, \mathcal{T}^\pi \eta') \leq \gamma^{1/2} \ell_{\xi_\pi,2}(\eta, \eta').$$

Lemma 9.15: Categorical Projection

For any signed distributions ν, ν' ,

$$\ell_2(\Pi_c \nu, \Pi_c \nu') \leq \ell_2(\nu, \nu').$$

Chapter 9 Lemmas: Feature Projection

The final projection is the analogue of the scalar linear projection, but applied to signed categorical distributions with the Cramér metric.

Lemma 9.16: Linear Feature Projection

For any $\eta, \eta' \in \mathcal{F}_{S,m}^{\mathcal{X}}$,

$$\ell_{\xi_{\pi},2}(\Pi_{\psi,\xi_{\pi},\ell_2}\eta, \Pi_{\psi,\xi_{\pi},\ell_2}\eta') \leq \ell_{\xi_{\pi},2}(\eta, \eta').$$

Thus each piece is either a contraction or a nonexpansion:

$\mathcal{T}^{\pi}$ contracts, $\Pi_c, \Pi_{\psi,\xi_{\pi},\ell_2}$ do not expand.

Chapter 9 Theorem: Doubly Projected Bellman

Composing the three preceding statements gives the actual convergence-style result for linear distributional approximation in the book.

Theorem 9.13

Under the assumptions above, for all $\eta, \eta' \in \mathcal{M}_{\ell_2,1}(\mathbb{R})^{\mathcal{X}}$,

$$\ell_{\xi_\pi,2}(\Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta,\Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta') \leq \gamma^{1/2}\ell_{\xi_\pi,2}(\eta,\eta').$$

Hence the projected distributional Bellman operator has a unique fixed point in $\mathcal{F}_{\psi,S,m}$, by Banach's fixed-point theorem.

Chapter 9: Exact Contraction Chain

For $\eta, \eta' \in \mathcal{M}_{\ell_2,1}(\mathbb{R})^{\mathcal{X}}$,

$$\begin{aligned} \ell_{\xi_\pi,2}(\Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta, \Pi_{\psi,\xi_\pi,\ell_2}\Pi_c\mathcal{T}^\pi\eta') \\ \leq \ell_{\xi_\pi,2}(\Pi_c\mathcal{T}^\pi\eta, \Pi_c\mathcal{T}^\pi\eta') \\ \leq \ell_{\xi_\pi,2}(\mathcal{T}^\pi\eta, \mathcal{T}^\pi\eta') \\ \leq \gamma^{1/2}\ell_{\xi_\pi,2}(\eta, \eta'). \end{aligned}$$

The three lines are exactly: feature projection nonexpansion, categorical projection nonexpansion, and distributional Bellman contraction.

Why Signed Measures Make the Theory Work

The scalar proof needs a linear subspace and an orthogonal projection. The signed categorical construction recreates that geometry:

$$p_i(x) \in \mathbb{R}, \quad \sum_i p_i(x) = 1.$$

Allowing negative masses is mathematically useful because affine combinations remain inside the signed unit-mass family.

Practical probability models lose this geometry

Softmax CTD/C51 enforces

$$p_i(x) \geq 0, \quad \sum_i p_i(x) = 1,$$

producing a nonlinear simplex-valued parameterization, not a linear projection space.

Quantile methods move atom locations instead of masses, so their projection is quantile regression, not an orthogonal projection in Cramér geometry.

Chapter 10: Why This Is Not a DQN Theorem

C51, QR-DQN, and IQN combine several ingredients:

nonlinear network + bootstrapping + greedy control + replay/target networks.

The theory is therefore much weaker than for policy evaluation.

What Theorem 9.13 needs

A fixed policy, linear projection in a fixed metric, and a fixed operator

$$\Pi_{\psi, \xi_{\pi}, \ell_2} \Pi_c \mathcal{T}^{\pi}.$$

What Chapter 10 uses

Nonlinear parameters and a moving control target, e.g.

$$r + \gamma Z_{\tilde{w}} \left(x', \arg \max_{a'} \mathbb{E}[Z_{\tilde{w}}(x', a')] \right).$$

So Chapter 10 inherits the projection intuition, but not a comparable general convergence theorem for deep distributional DQN.

From Fixed Features to Learned Features

A deep network replaces the hand-designed map $\phi(x)$ by a learned representation. Its penultimate layer can still be read as

$$\phi_w(x) \in \mathbb{R}^n,$$

followed by linear prediction heads.

- Earlier layers learn which state distinctions matter.
- Final layers map those features to values or distribution parameters.
- Backpropagation updates both simultaneously.

Linear approximation is therefore not discarded; it remains the final layer and the conceptual model for the deep semi-gradient update.

DQN: Prediction, Behavior, and Experience

DQN maps an observation x to one value per action:

$$Q_w(x, a), \quad a \in \mathcal{A}.$$

In the canonical Atari setup:

- four preprocessed 84×84 frames form the network input;
- convolutional layers and a fully connected layer produce 512 features;
- a linear output layer produces $|\mathcal{A}|$ action values;
- an ε -greedy policy acts from Q_w ;
- transitions are stored and resampled from replay memory.

DQN: Semi-Gradient Learning

With online weights w , target weights $\tilde{w}$, and transition (x, a, r, x') , define

$$y = r + \gamma \max_{a' \in \mathcal{A}} Q_{\tilde{w}}(x', a').$$

The squared loss and semi-gradient update are

$$L(w) = (y - Q_w(x, a))^2,$$

$$w \leftarrow w + \alpha(y - Q_w(x, a))\nabla_w Q_w(x, a).$$

If $Q_w(x, a) = \phi(x, a)^\top w$, then $\nabla_w Q_w = \phi$: this is exactly the linear update. Replay reduces temporal correlation; the frozen target network slows the moving-target feedback loop.

Distributional Network Output

Deep distributional RL changes the output from one scalar per action to m distribution parameters per action:

$$x \longmapsto \mathbb{R}^{|\mathcal{A}| \times m}.$$

The interpretation determines the algorithm:

- **C51**: fixed locations, learned softmax probabilities;
- **QR-DQN**: equal probabilities, learned quantile locations;
- **IQN**: quantile level τ becomes an additional input.

Behavior is usually risk-neutral: derive $Q_w(x, a)$ from the predicted distribution and do ε -greedy action selection as in DQN.

C51: Prediction and Behavior

For fixed $\theta_1 < \dots < \theta_m$, the network outputs logits $\varphi_i(x, a; w)$ and

$$p_i(x, a; w) = \frac{e^{\varphi_i(x, a; w)}}{\sum_{j=1}^m e^{\varphi_j(x, a; w)}}, \quad \eta_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \delta_{\theta_i}.$$

The induced action value is

$$Q_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \theta_i.$$

Original C51 uses $m = 51$ evenly spaced locations; support choice imposes an important inductive bias because projected targets outside it are clipped.

C51: Bellman Target and Cross-Entropy

Select the target-network greedy action

$$a_{\tilde{w}}(x') \in \arg \max_{a'} Q_{\tilde{w}}(x', a').$$

Construct and project the target

$$\bar{\eta}(x, a) = \Pi_c(b_{r,\gamma})_{\#} \eta_{\tilde{w}}(x', a_{\tilde{w}}(x')) = \sum_{j=1}^m \bar{p}_j \delta_{\theta_j}.$$

Then minimize

$$L(w) = - \sum_{j=1}^m \bar{p}_j \log p_j(x, a; w).$$

This is the nonlinear analogue of linear CTD, except shared early-layer weights influence all particles and cannot be updated independently.

QR-DQN: Quantile Prediction

QR-DQN interprets the m outputs as locations of an equally weighted quantile distribution:

$$\eta_w(x, a) = \frac{1}{m} \sum_{i=1}^m \delta_{\theta_i(x, a; w)}, \quad Q_w(x, a) = \frac{1}{m} \sum_{i=1}^m \theta_i(x, a; w).$$

With $a_{\tilde{w}}(x')$ greedy by this mean, the pairwise residuals are

$$\Delta_{ij} = r + \gamma \theta_j(x', a_{\tilde{w}}(x'); \tilde{w}) - \theta_i(x, a; w).$$

Every predicted quantile is compared with every target quantile.

QR-DQN: Quantile Huber Loss

For $\tau_i = (2i - 1)/(2m)$, QR-DQN minimizes

$$L(w) = \frac{1}{m} \sum_{i,j=1}^m \rho_{\tau_i}^H(\Delta_{ij}), \quad \rho_{\tau}^H(u) = |\mathbb{1}_{\{u < 0\}} - \tau| H_1(u),$$

where

$$H_1(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq 1, \\ |u| - \frac{1}{2}, & |u| > 1. \end{cases}$$

The quantile asymmetry places different heads at different parts of the distribution; the Huber transition gives smooth small-error gradients and limits the influence of large errors.

IQN: Make the Quantile Level an Input

C51 and QR-DQN output a fixed finite parameter vector. IQN instead models the quantile function itself:

$$(x, a, \tau) \mapsto \theta_w(x, a, \tau) \approx F_{\eta(x,a)}^{-1}(\tau).$$

The level is encoded by

$$\varphi(\tau) = (\cos(\pi \tau i))_{i=0}^{M-1},$$

transformed by a fully connected layer, and combined multiplicatively with the state features. The final action heads satisfy

$$\theta_w(x, a, \tau) = \phi_w(x, \tau)^\top w_a.$$

IQN: Behavior and Learning

Approximate the mean with independent $\tau_i \sim \mathcal{U}(0, 1)$:

$$Q_w(x, a) \approx \frac{1}{m} \sum_{i=1}^m \theta_w(x, a, \tau_i).$$

For independent online levels τ_i and target levels τ'_j , minimize pairwise losses of

$$r + \gamma \theta_{\tilde{w}}(x', a_{\tilde{w}}(x'), \tau'_j) - \theta_w(x, a, \tau_i).$$

Sampling many level pairs reduces gradient variance. IQN approximates the whole quantile function without committing to fixed output locations.

IQN Makes Risk Sensitivity Simple

Risk-neutral behavior samples $\tau \sim \mathcal{U}(0, 1)$. Lower-tail CVaR at level $\bar{\tau}$ is

$$\text{CVaR}_{\bar{\tau}}(G_w(x, a)) = \frac{1}{\bar{\tau}} \int_0^{\bar{\tau}} \theta_w(x, a, \tau) d\tau.$$

Approximate it by changing only the sampling distribution:

$$\text{CVaR}_{\bar{\tau}}(G_w(x, a)) \approx \frac{1}{m} \sum_{i=1}^m \theta_w(x, a, \tau_i), \quad \tau_i \sim \mathcal{U}(0, \bar{\tau}).$$

Thus an implicit representation separates learning the return distribution from choosing how that distribution should guide behavior.

How Distributional Predictions Shape Features

Split the network into learned features and final prediction heads.

- DQN trains the representation to predict one conditional mean per action.
- C51 trains it through m probability errors per action.
- QR-DQN/IQN train it through many pairwise quantile residuals.

The type and number of predictions therefore change the learned state representation. This richer auxiliary prediction problem is a leading explanation for improved control performance even when actions are ultimately selected by the mean.

Takeaways

- ➊ Linear approximation introduces state aliasing and a weighted projection problem; the sampling distribution is central to stability.
- ➋ Semi-gradient TD tracks a projected Bellman process, and target weights make that relationship explicit.
- ➌ Linear QTD and CTD make distribution *parameters* linear in features.
- ➍ Deep methods learn the features and retain the same prediction–target–semi-gradient structure.
- ➎ C51, QR-DQN, and IQN differ mainly in how they parameterize and train the output distribution; that choice also shapes the learned representation.

References

- Bellemare, Dabney, Rowland (2023): *Distributional Reinforcement Learning*, Chapters 9–10.
- Bellemare, Dabney, Munos (2017): A Distributional Perspective on Reinforcement Learning.
- Dabney et al. (2018): Distributional Reinforcement Learning with Quantile Regression.
- Dabney et al. (2018): Implicit Quantile Networks for Distributional Reinforcement Learning.